```
from datetime import date

students = {}
attendance_records = []

INSTRUCTOR_USERNAME = "Instructor"
INSTRUCTOR_PASSWORD = "1234"


def print_header(title, width=80):
    print("=" * width)
    print(title.center(width))
    print("=" * width)


def today_date():
    return date.today().strftime("%d/%m/%Y")


def valid_school_id(school_id):
    parts = school_id.split("-")
    return (
        len(parts) == 3
        and len(parts[0]) == 2
        and parts[0].isdigit()
        and len(parts[1]) == 4
        and parts[1].isdigit()
        and len(parts[2]) == 6
        and parts[2].isdigit()
    )


def valid_full_name(name):
    parts = name.strip().split()

    if len(parts) < 2:
        return False

    for part in parts:
        cleaned = (
            part.replace(".", "")
            .replace("-", "")
            .replace("'", "")
        )
```

```
        if not cleaned.isalpha():
            return False

        if "." in part and len(part.replace(".", "")) != 1:
            return False

    return True


def confirm_action():
    while True:
        answer = input("Confirm? (Y/N): ").strip().upper()

        if answer in ("Y", "N"):
            return answer == "Y"

        print("Invalid Input. Please enter Y or N.")


def display_registered_students():
    if not students:
        print("\nNo registered students found.")
        return

    print("\n" + "-" * 80)
    print(f"{'No.':<6}{'School ID':<22}{'Full Name':<52}")
    print("-" * 80)

    sorted_ids = sorted(
        students,
        key=lambda sid: students[sid]["name"].casefold()
    )

    for number, school_id in enumerate(sorted_ids, start=1):
        print(
            f"{number:<6}{school_id:<22}"
            f"{students[school_id]['name']:<52}"
        )

    print("-" * 80)


def get_school_id(prompt="Enter School ID (0 to return): "):
    while True:
        school_id = input(prompt).strip()
```

```
        if school_id == "0":
            return "0"

        if not valid_school_id(school_id):
            print(
                "Invalid School ID. "
                "Use the format 02-2627-123456."
            )
            continue

        return school_id


def register_students():
    while True:
        print_header("STUDENT REGISTRATION")

        name = input(
            "Enter Full Name (0 to return): "
        ).strip()

        if name == "0":
            return

        if not valid_full_name(name):
            print(
                "Invalid Input. "
                "Please enter a valid full name."
            )
            continue

        school_id = get_school_id()

        if school_id == "0":
            return

        if school_id in students:
            print("School ID already registered.")
            continue

        print(f"\nFull Name : {name}")
        print(f"School ID : {school_id}")

        if confirm_action():
```

```
            students[school_id] = {
                "name": name,
                "attendance": []
            }
            print("Student successfully registered.")
        else:
            print("Registration cancelled.")

        while True:
            answer = input(
                "Register another student? (Y/N): "
            ).strip().upper()

            if answer == "Y":
                break

            if answer == "N":
                return

            print("Invalid Input. Please enter Y or N.")


def view_registered_students():
    print_header("REGISTERED STUDENTS")
    display_registered_students()


def edit_student_details():
    while True:
        print_header("EDIT STUDENT DETAILS")
        display_registered_students()

        if not students:
            return

        choice = input(
            "Enter student number to edit (0 to return): "
        ).strip()

        if choice == "0":
            return

        ids = sorted(
            students.keys(),
            key=lambda sid: students[sid]["name"].casefold()
```

```
)

if not choice.isdigit() or not (
    1 <= int(choice) <= len(ids)
):
    print("Invalid student number.")
    continue

old_id = ids[int(choice) - 1]

print(
    f"Selected: {students[old_id]['name']} "
    f"({old_id})"
)

print(
    "1. Edit Full Name\n"
    "2. Edit School ID\n"
    "0. Return"
)

edit_choice = input("Enter your choice: ").strip()

if edit_choice == "0":
    continue

if edit_choice == "1":
    new_name = input(
        "Enter New Full Name: "
    ).strip()

    if not valid_full_name(new_name):
        print(
            "Invalid Input. "
            "Please enter a valid full name."
        )
        continue

    print(f"New name: {new_name}")

    if confirm_action():
        students[old_id]["name"] = new_name

        for record in attendance_records:
            if record["school_id"] == old_id:
```

```
                        record["name"] = new_name

                print("Student details successfully updated.")

        elif edit_choice == "2":
            new_id = get_school_id(
                "Enter New School ID (0 to cancel): "
            )

            if new_id == "0":
                continue

            if new_id != old_id and new_id in students:
                print("School ID already registered.")
                continue

            if confirm_action():
                data = students.pop(old_id)
                students[new_id] = data

                for record in attendance_records:
                    if record["school_id"] == old_id:
                        record["school_id"] = new_id

                print("Student details successfully updated.")

        else:
            print("Invalid choice.")


def get_attendance_status():
    while True:
        status = input(
            "Enter status "
            "(P=Present, A=Absent, L=Late, E=Excused): "
        ).strip().upper()

        if status == "P":
            return "Present"

        if status == "A":
            return "Absent"

        if status == "L":
            return "Late"
```

```
        if status == "E":
            return "Excused"

        print("Invalid Input. Please enter P, A, L, or E.")


def record_normal_attendance():
    if not students:
        print("No registered students available.")
        return

    previous_sessions = [
        record["session"]
        for record in attendance_records
        if record["type"] == "Normal Attendance"
        and record["session"].startswith("Attendance ")
    ]

    highest_number = 0

    for session_name in previous_sessions:
        number_text = session_name.replace(
            "Attendance ", "", 1
        )

        if number_text.isdigit():
            highest_number = max(
                highest_number,
                int(number_text)
            )

    session = f"Attendance {highest_number + 1}"
    record_date = today_date()
    new_records = []

    print_header(session.upper())
    print(f"Date: {record_date} (DD/MM/YYYY)")

    sorted_students = sorted(
        students.items(),
        key=lambda item: item[1]["name"].casefold()
    )

    for school_id, student in sorted_students:
```

```
    print(
        f"\nStudent: {student['name']} "
        f"| School ID: {school_id}"
    )

    status = get_attendance_status()

    new_records.append({
        "type": "Normal Attendance",
        "session": session,
        "date": record_date,
        "school_id": school_id,
        "name": student["name"],
        "status": status,
        "completion": None,
        "activity_points": 0.0,
        "makeup_status": None,
        "makeup_points": 0.0
    })

print("\nAttendance Preview")

for record in sorted(
    new_records,
    key=lambda item: item["name"].casefold()
):
    print(
        f"{record['school_id']} | "
        f"{record['name']} | "
        f"{record['status']} | "
        f"{record['date']}"
    )

if confirm_action():
    attendance_records.extend(new_records)

    for record in new_records:
        students[record["school_id"]]["attendance"].append(
            record
        )

    print("Normal attendance successfully recorded.")
else:
    print("Normal attendance cancelled.")
```

```
def record_activity_attendance():
    if not students:
        print("No registered students available.")
        return

    activity = input(
        "Enter Activity / Module Name "
        "(e.g., M1: Activity 1): "
    ).strip()

    if not activity:
        print("Invalid Input. Activity name cannot be empty.")
        return

    record_date = today_date()
    new_records = []

    print_header("ACTIVITY-BASED ATTENDANCE")
    print(f"Activity: {activity}")
    print(f"Date: {record_date} (DD/MM/YYYY)")

    sorted_students = sorted(
        students.items(),
        key=lambda item: item[1]["name"].casefold()
    )

    for school_id, student in sorted_students:
        print(
            f"\nStudent: {student['name']} "
            f"| School ID: {school_id}"
        )

        status = get_attendance_status()

        if status == "Present":
            completion = "Completed"
            points = 1.0
            makeup = "Not Needed"

        elif status == "Late":
            completion = "Completed"
            points = 0.5
            makeup = "Not Needed"
```

```
    else:
        completion = "Missed Activity"
        points = 0.0
        makeup = "Not Yet Completed"

    new_records.append({
        "type": "Activity-Based Attendance",
        "session": activity,
        "date": record_date,
        "school_id": school_id,
        "name": student["name"],
        "status": status,
        "completion": completion,
        "activity_points": points,
        "makeup_status": makeup,
        "makeup_points": 0.0
    })

print("\nActivity Attendance Preview")

for record in sorted(
    new_records,
    key=lambda item: item["name"].casefold()
):
    print(
        f"{record['school_id']} | "
        f"{record['name']} | "
        f"{record['status']} | "
        f"{record['completion']} | "
        f"{record['date']} | "
        f"{record['session']}"
    )

if confirm_action():
    attendance_records.extend(new_records)

    for record in new_records:
        students[record["school_id"]]["attendance"].append(
            record
        )

    print("Activity-based attendance successfully recorded.")
else:
    print("Activity-based attendance cancelled.")
```

```
def record_class_attendance():
    while True:
        print_header("RECORD CLASS ATTENDANCE")
        print(
            "1. Normal Attendance\n"
            "2. Activity-Based Attendance\n"
            "0. Return"
        )

        choice = input("Enter your choice: ").strip()

        if choice == "1":
            record_normal_attendance()
        elif choice == "2":
            record_activity_attendance()
        elif choice == "0":
            return
        else:
            print("Invalid choice.")


def calculate_summary(records):
    present = sum(
        1 for record in records
        if record["status"] == "Present"
    )

    late = sum(
        1 for record in records
        if record["status"] == "Late"
    )

    absent = sum(
        1 for record in records
        if record["status"] == "Absent"
    )

    excused = sum(
        1 for record in records
        if record["status"] == "Excused"
    )

    counted = present + late + absent
    points = present + late * 0.5
```

```
    rate = points / counted * 100 if counted else 0

    return {
        "present": present,
        "late": late,
        "absent": absent,
        "excused": excused,
        "points": points,
        "total_counted": counted,
        "rate": rate
    }


def display_summary(records):
    summary = calculate_summary(records)

    print(f"Present : {summary['present']}")
    print(f"Late    : {summary['late']}")
    print(f"Absent  : {summary['absent']}")
    print(f"Excused : {summary['excused']}")
    print(f"Points  : {summary['points']}")
    print(f"Attendance Rate: {summary['rate']:.2f}%")

    if (
        summary["total_counted"]
        and summary["rate"] < 75
    ):
        print("WARNING: Attendance is below 75%.")


def get_missed_activities(records):
    return [
        record for record in records
        if record["type"] == "Activity-Based Attendance"
        and record["status"] in ("Absent", "Excused")
        and record.get("makeup_status") != "Completed"
    ]
def manage_makeup_activities():
    while True:
        print_header("RECORD ACTIVITY MAKE-UP")
        items = []

        sorted_students = sorted(
            students.items(),
            key=lambda item: item[1]["name"].casefold()
```

```
)

for school_id, student in sorted_students:
    for record in student["attendance"]:
        if (
            record["type"] == "Activity-Based Attendance"
            and record["status"] in ("Absent", "Excused")
            and record.get("makeup_status") != "Completed"
        ):
            items.append((school_id, student, record))

if not items:
    print("There are no pending missed activities.")
    return

print(
    f"{'No.':<5}{'School ID':<20}"
    f"{'Student':<25}{'Activity':<35}"
    f"{'Status':<12}{'Date':<12}"
)
print("-" * 109)

for number, (school_id, student, record) in enumerate(
    items, start=1
):
    print(
        f"{number:<5}{school_id:<20}"
        f"{student['name']:<25}"
        f"{record['session']:<35}"
        f"{record['status']:<12}"
        f"{record['date']:<12}"
    )

choice = input(
    "Enter item number to update (0 to return): "
).strip()

if choice == "0":
    return

if not choice.isdigit() or not (
    1 <= int(choice) <= len(items)
):
    print("Invalid choice.")
    continue
```

```
school_id, student, record = items[int(choice) - 1]

print(f"\nStudent: {student['name']}")
print(f"Activity: {record['session']}")
print(
    f"Original status: {record['status']} "
    f"| Original date: {record['date']}"
)

print(
    "1. Mark Make-Up Completed\n"
    "2. Keep Not Yet Completed\n"
    "0. Cancel"
)

action = input("Enter your choice: ").strip()

if action == "1":
    print(
        "Make-up result: Completed Late "
        "| Points: 0.25"
    )

    if confirm_action():
        record["makeup_status"] = "Completed"
        record["makeup_date"] = today_date()
        record["makeup_points"] = 0.25
        record["activity_points"] = 0.25
        record["completion"] = "Completed Late"

        print(
            "Make-up activity successfully updated."
        )
    else:
        print("Update cancelled.")

elif action == "2":
    print(
        "The activity remains Not Yet Completed."
    )

elif action != "0":
    print("Invalid choice.")
```

```
def print_activity_summary(records):
    activity_records = [
        record for record in records
        if record["type"] == "Activity-Based Attendance"
    ]

    if not activity_records:
        print("No activity-based attendance records found.")
        return

    on_time = sum(
        1 for record in activity_records
        if record["completion"] == "Completed"
    )

    late = sum(
        1 for record in activity_records
        if record["completion"] == "Completed Late"
    )

    missed = get_missed_activities(activity_records)

    points = sum(
        record.get("activity_points", 0.0)
        for record in activity_records
    )

    print(f"Completed on time       : {on_time}")
    print(f"Completed late (make-up): {late}")
    print(f"Missed Activities       : {len(missed)}")
    print(f"Total Activity Points   : {points:.2f}")

    if missed:
        print("\nMissed Activities:")

        # Sort by student name only.
        # Keep activity names in their original recorded order.
        for record in sorted(
            missed,
            key=lambda item: item["name"].casefold()
        ):
            print(record["date"])
            print(record["name"])
            print(
```

```
                f"{record['session']}; "
                f"Status: {record['status']}"
            )


def print_student_report(student, school_id):
    normal = [
        record for record in student["attendance"]
        if record["type"] == "Normal Attendance"
    ]

    activities = [
        record for record in student["attendance"]
        if record["type"] == "Activity-Based Attendance"
    ]

    print("\n" + "=" * 80)
    print("STUDENT INFORMATION".center(80))
    print("=" * 80)

    print(f"Full Name : {student['name']}")
    print(f"School ID : {school_id}")

    print("\nNORMAL ATTENDANCE HISTORY")
    print(
        f"{'Session':<22}{'Date':<13}"
        f"{'Status':<18}{'Points':<10}"
    )
    print("-" * 65)

    if not normal:
        print("No normal attendance records found.")

    for record in normal:
        points = (
            1 if record["status"] == "Present"
            else 0.5 if record["status"] == "Late"
            else 0
        )

        print(
            f"{record['session']:<22}"
            f"{record['date']:<13}"
            f"{record['status']:<18}"
            f"{points:<10}"
```

```
        )

    print("\nNormal Attendance Summary")
    display_summary(normal)

    print("\nACTIVITY-BASED ATTENDANCE HISTORY")
    print(
        f"{'Activity / Module':<35}{'Date':<13}"
        f"{'Status':<13}{'Completion':<22}"
        f"{'Points':<9}{'Make-Up':<18}"
    )
    print("-" * 110)

    if not activities:
        print("No activity-based attendance records found.")

    for record in activities:
        print(
            f"{record['session']:<35}"
            f"{record['date']:<13}"
            f"{record['status']:<13}"
            f"{record['completion']:<22}"
            f"{record.get('activity_points', 0.0):<9.2f}"
            f"{record.get('makeup_status', 'Not Needed'):<18}"
        )

        if record.get("makeup_status") == "Completed":
            print(
                f"  Make-up completed on: "
                f"{record.get('makeup_date', 'Date not available')} "
                f"| Make-up points: "
                f"{record.get('makeup_points', 0.25):.2f}"
            )

    print("\nActivity-Based Attendance Summary")
    print_activity_summary(activities)


def view_individual_attendance_summary():
    while True:
        print_header("INDIVIDUAL ATTENDANCE SUMMARY")
        display_registered_students()

        if not students:
            return
```

```
        school_id = get_school_id(
            "Enter School ID (0 to return): "
        )

        if school_id == "0":
            return

        if school_id not in students:
            print("Student not found.")
            continue

        print_student_report(
            students[school_id],
            school_id
        )

        input("\nPress Enter to continue...")


def view_consolidated_class_attendance():
    print_header("CONSOLIDATED CLASS ATTENDANCE")

    if not students:
        print("No registered students found.")
        return

    normal = [
        record for record in attendance_records
        if record["type"] == "Normal Attendance"
    ]

    activities = [
        record for record in attendance_records
        if record["type"] == "Activity-Based Attendance"
    ]

    print("\nA. NORMAL ATTENDANCE")
    print(
        f"{'Session':<22}{'Date':<13}"
        f"{'School ID':<20}{'Full Name':<30}"
        f"{'Status':<13}"
    )
    print("-" * 98)
```

```
if not normal:
    print("No normal attendance records found.")

# Alphabetize by student name only.
for record in sorted(
    normal,
    key=lambda item: item["name"].casefold()
):
    print(
        f"{record['session']:<22}"
        f"{record['date']:<13}"
        f"{record['school_id']:<20}"
        f"{record['name']:<30}"
        f"{record['status']:<13}"
    )

print("\nNormal Attendance Summary")
display_summary(normal)

print("\nB. ACTIVITY-BASED ATTENDANCE")
print(
    f"{'Activity':<35}{'Date':<13}"
    f"{'School ID':<20}{'Full Name':<27}"
    f"{'Status':<13}{'Completion':<20}"
)
print("-" * 128)

if not activities:
    print("No activity-based attendance records found.")

# Alphabetize by student name only.
# Do not sort or shorten the activity labels.
for record in sorted(
    activities,
    key=lambda item: item["name"].casefold()
):
    print(
        f"{record['session']:<35}"
        f"{record['date']:<13}"
        f"{record['school_id']:<20}"
        f"{record['name']:<27}"
        f"{record['status']:<13}"
        f"{record['completion']:<20}"
    )
```

```
print("\nActivity-Based Attendance Summary")

# Keep activities in the order they were first recorded.
activity_names = []

for record in activities:
    if record["session"] not in activity_names:
        activity_names.append(record["session"])

for activity in activity_names:
    group = [
        record for record in activities
        if record["session"] == activity
    ]

    on_time = sum(
        1 for record in group
        if record["completion"] == "Completed"
    )

    makeup = sum(
        1 for record in group
        if record["completion"] == "Completed Late"
    )

    missed_count = len(get_missed_activities(group))

    points = sum(
        record.get("activity_points", 0.0)
        for record in group
    )

    print(f"\nActivity: {activity}")
    print(f"Completed on time       : {on_time}")
    print(f"Completed late (make-up): {makeup}")
    print(f"Missed Activities       : {missed_count}")
    print(f"Total Activity Points   : {points:.2f}")

all_missed = get_missed_activities(activities)

if all_missed:
    print("\nStudents with Missed Activities:")

    # Sort by student name only.
    # The activity name is displayed in full as recorded.
```

```
        for record in sorted(
            all_missed,
            key=lambda item: item["name"].casefold()
        ):
            print(record["date"])
            print(record["name"])
            print(
                f"{record['session']}; "
                f"Status: {record['status']}"
            )

    print(
        "\nMake-up points affect activity-based points only, "
        "not normal attendance percentage."
    )
def instructor_menu():
    while True:
        print_header("INSTRUCTOR MENU")

        print("1. Register Students")
        print("2. View Registered Students")
        print("3. Edit Student Details")
        print("4. Record Class Attendance")
        print("5. View Individual Attendance Summary")
        print("6. View Consolidated Class Attendance")
        print("7. Record Activity Make-Up")
        print("8. Logout")

        choice = input("Enter your choice: ").strip()

        if choice == "1":
            register_students()

        elif choice == "2":
            view_registered_students()

        elif choice == "3":
            edit_student_details()

        elif choice == "4":
            record_class_attendance()

        elif choice == "5":
            view_individual_attendance_summary()
```

```
        elif choice == "6":
            view_consolidated_class_attendance()

        elif choice == "7":
            manage_makeup_activities()

        elif choice == "8":
            print("Instructor logged out.")
            return

        else:
            print("Invalid choice.")


def instructor_login():
    print_header("INSTRUCTOR LOGIN")

    username = input("Enter Username: ").strip()

    if username != INSTRUCTOR_USERNAME:
        print("Invalid Username")
        return

    password = input("Enter Password: ").strip()

    if password != INSTRUCTOR_PASSWORD:
        print("Wrong Password")
        return

    print("Login successful.")
    instructor_menu()


def student_access():
    print_header("STUDENT ACCESS")

    school_id = get_school_id(
        "Enter your School ID (0 to return): "
    )

    if school_id == "0":
        return

    if school_id not in students:
        print("Student not found.")
```

```
        return

    print_student_report(
        students[school_id],
        school_id
    )

    input("\nPress Enter to return to the main menu...")


def main():
    while True:
        print_header("ATTENDEASE")

        print(
            "A Console-Based Attendance Tracking "
            "and Data Entry System"
        )

        print(
            "\n1. Instructor Access"
            "\n2. Student Access"
            "\n0. Exit"
        )

        choice = input("Enter your choice: ").strip()

        if choice == "1":
            instructor_login()

        elif choice == "2":
            student_access()

        elif choice == "0":
            print("Thank you for using AttendEase.")
            break

        else:
            print("Invalid choice.")


main()
```